Signals and Systems Lab 10

Demarce Williams DSW9740

Lab 10: Echo Cancellation

```
% Clear workspace and command window
clear; close all; clc;
```

PART 1: Create the Echo Signal

```
% Load the 'matlab' speech signal
load mtlb; % Loads 'mtlb' (speech signal) and 'Fs' (sampling frequency)
x = mtlb; % Rename for convenience

% Plays sound
soundsc(x,Fs);

Nsamples = length(x);
t_x = (0:Nsamples-1)/Fs; % Time axis vector in seconds

disp('1. & 2. Signal Loading and Duration');
```

```
1. & 2. Signal Loading and Duration
```

```
disp(['Sampling Frequency (Fs): ', num2str(Fs), ' Hz']);
```

```
Sampling Frequency (Fs): 7418 Hz
```

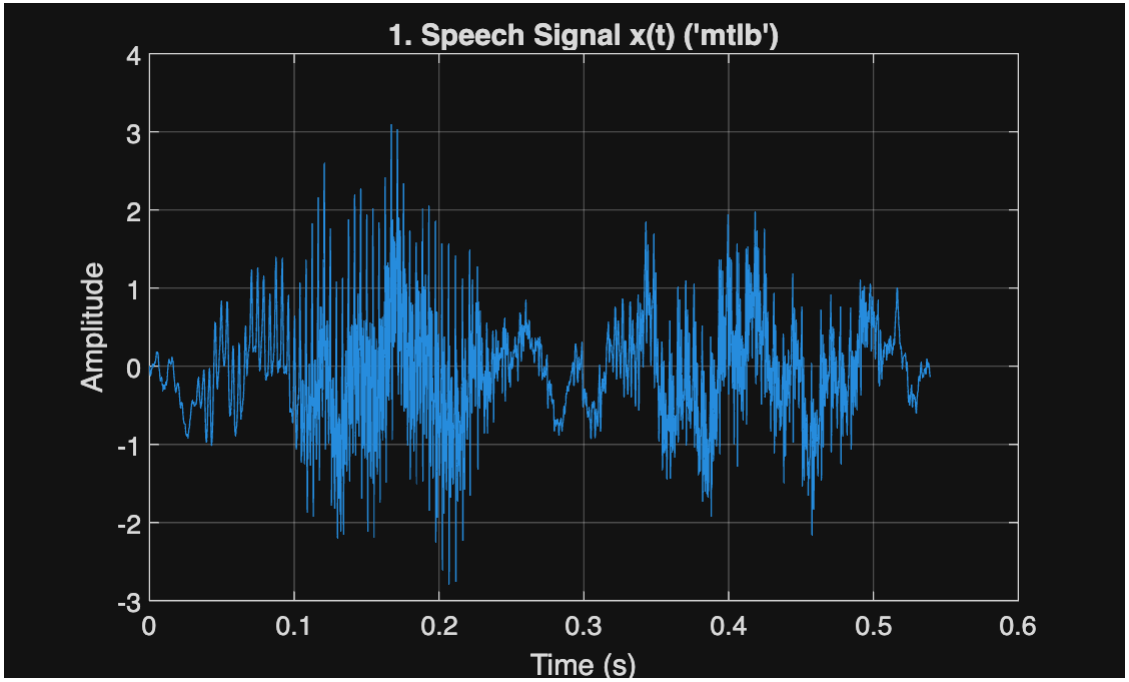
```

% Exact Duration of the signal
duration_seconds = Nsamples/Fs;
disp(['Signal Duration (length(x)/Fs): ', num2str(duration_seconds), ' seconds']);
```

```
Signal Duration (length(x)/Fs): 0.53936 seconds
```

```
disp(['Signal Length (Nsamples): ', num2str(Nsamples), ' samples']);
```

```
Signal Length (Nsamples): 4001 samples
```

```
% Plot the original signal x(t)
figure;
plot(t_x, x);
xlabel('Time (s)');
ylabel('Amplitude');
title('1. Speech Signal x(t) (''mtlb'')');
grid on;
```




```
syms n z alpha N_sym real;
disp(' ');
```

```
disp('3. Impulse Response of the Echo System');
```

```
3. Impulse Response of the Echo System
```

```
% The impulse response h(n) of the system y(n) = x(n) + alpha*x(n-N) is:
%h(n) = delta(n) + alpha*delta(n-N)
h_sym = dirac(n) + alpha*dirac(n-N_sym);
disp('Impulse Response h[n] = ');disp(h_sym);
```

```
Impulse Response h[n] =
```

$$\delta(n) + \alpha\,\delta(N_{\mathrm{sym}} - n)$$

```
% Z-transform
H_sym = 1 + alpha*z^(-N_sym);
disp('Z-Transform of Impulse Response, H[z] = ');
```

```
Z-Transform of Impulse Response, H[z] =
```

```
disp(H_sym);
```

$$\frac{\alpha}{z^{N_{\mathrm{sym}}}} + 1$$

```
disp('Delay explanation: t0 = N/Fs because Fs is samples/second, so N samples / Fs (samples/sec) yields
time in seconds.');
```

```
Delay explanation: t0 = N/Fs because Fs is samples/second, so N samples / Fs (samples/sec) yields time in seconds.
```

```
% 4. Define Echo Parameters and Calculate Delay N
t0 = 0.1;       % Delay in seconds
alpha = 0.9;    % Scaling amplitude of echo
% Calculate N
N = round(t0*Fs); % Number of delayed samples

disp(' ');
```

```
disp(' 4. Echo Parameters and Convolution');
```

```
 4. Echo Parameters and Convolution
```

```
disp(['Desired delay t0: ', num2str(t0), ' seconds']);
```

```
Desired delay t0: 0.1 seconds
```

```
disp(['Echo amplitude alpha: ', num2str(alpha)]);
```

```
Echo amplitude alpha: 0.9
```

```
disp(['Calculated delay N (t0 * Fs): ', num2str(N), ' samples']);
```

```
Calculated delay N (t0 * Fs): 742 samples
```

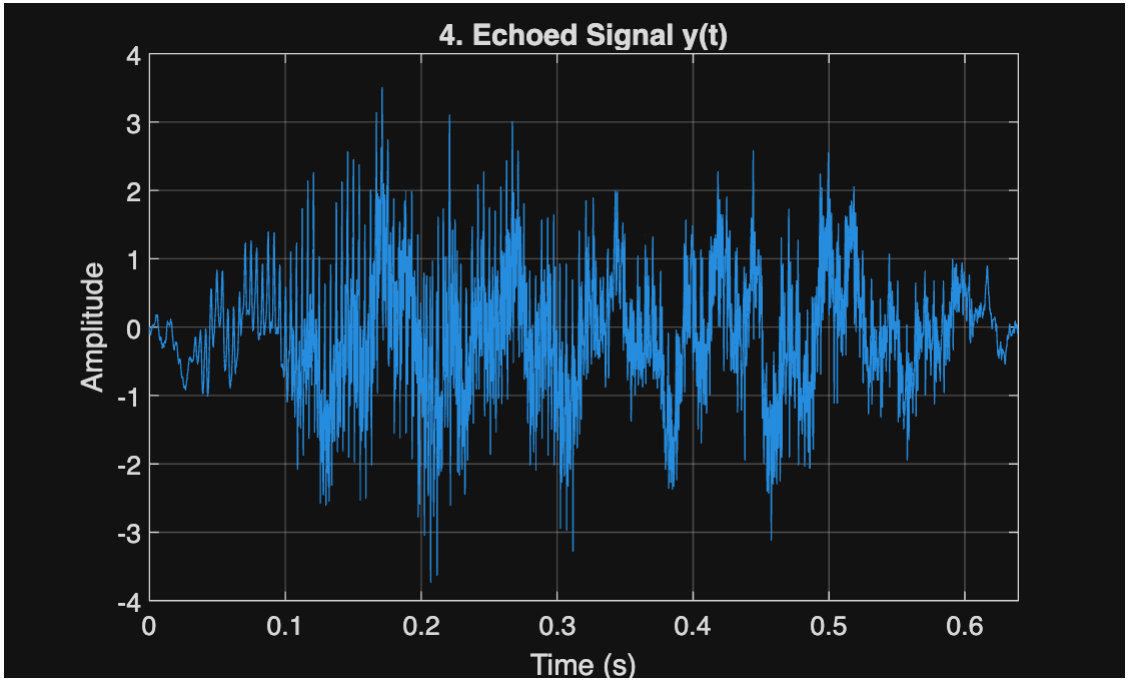
```

% Construct the impulse response vector h
% h superimposes an echo on the original signal when convolved.
h = zeros(1, N+1);
h(1)   = 1;       % delta[n] term
h(N+1) = alpha;   % alpha*delta[n-N] term (N+1 since indexing starts at 1)

disp(['Impulse response vector h has length: ', num2str(length(h))]);
```

```
Impulse response vector h has length: 743
```

```

% Generate echoed signal y
y = conv(h, x);
t_y = (0:length(y)-1)/Fs; % Time axis for y

% Play sound
soundsc(y, Fs);
disp('The signal y(n) has been created and is plotted below.');
```

```
The signal y(n) has been created and is plotted below.
```

```

% Plot echoed signal
figure;
plot(t_y, y);
xlabel('Time (s)');
ylabel('Amplitude');
title('4. Echoed Signal y(t)');
xlim([0 min(0.7, t_y(end))]);
grid on;
```




```
disp(' ');
```

```
disp('5. Inverse System Impulse Response');
```

```
5. Inverse System Impulse Response
```

```
% 5. The inverse system transfer function is G(z) = 1 / (1 + alpha*z^(-N))
% This can be expanded as a geometric series (for |alpha*z^(-N)| < 1):
% G(z) = 1 - alpha*z^(-N) + alpha^2*z^(-2N) - alpha^3*z^(-3N) + ...
G_sym = simplify(1/H_sym);
disp('Z-Transform of Inverse System G(z) = ');disp(G_sym);
```

```
Z-Transform of Inverse System G(z) =
```

$$\frac{1}{\frac{\alpha}{z^{N_{\mathrm{sym}}}} + 1}$$

```
syms k; g_sym = symsum(((-alpha)^k) * dirac(n - k*N_sym), k, 0, inf);
```

```
disp('Impulse Response g[n] = ');disp(g_sym);
```

```
Impulse Response g[n] =
```

$$\sum_{k=0}^{\infty} \left(-\frac{9}{10}\right)^{k} \delta(n - N_{\mathrm{sym}}\, k)$$

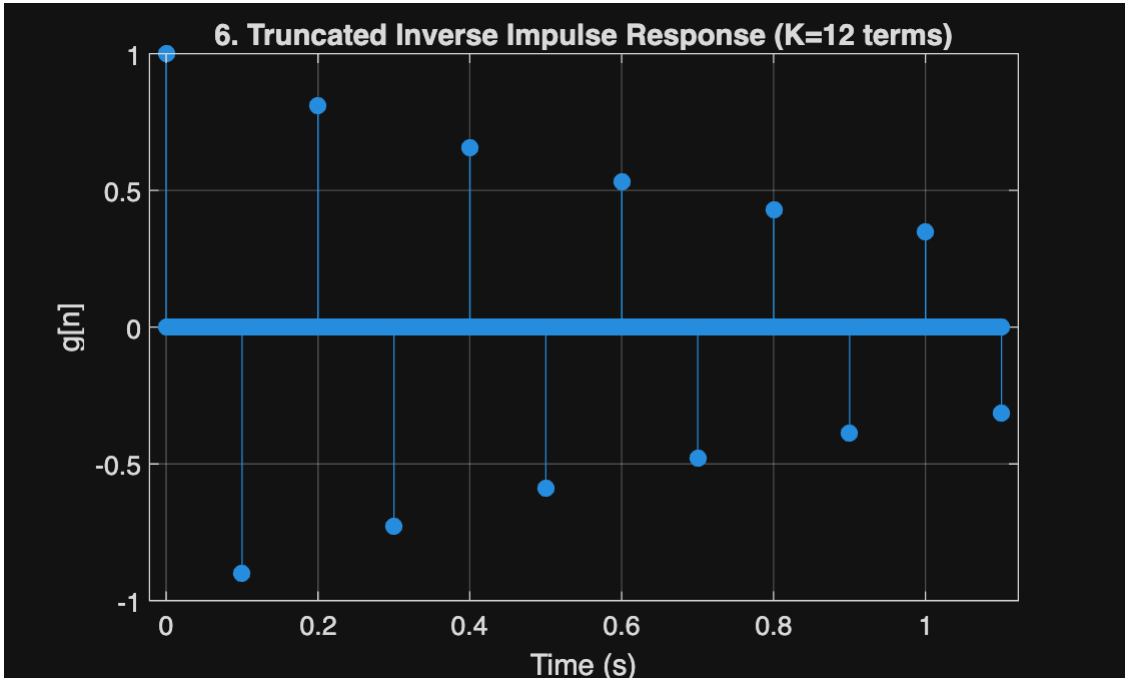
```
disp('Duration of the inverse impulse response is: Infinite (IIR filter).');
```

```
Duration of the inverse impulse response is: Infinite (IIR filter).
```

```

```

PART 2: Remove the Echo (Known Parameters)

```
% 6. Plot the Truncated Inverse Impulse Response
K = 12; % Number of terms to show (truncation)
g_len = (K-1)*N + 1;
g = zeros(1, g_len);

for k = 0:K-1
    idx = 1 + k*N;
    g(idx) = (-alpha)^k;
end
t_g = (0:g_len-1)/Fs; % Time axis in seconds

figure;
stem(t_g, g, 'filled');
xlabel('Time (s)');
ylabel('g[n]');
title(['6. Truncated Inverse Impulse Response (K=', num2str(K), ' terms)']);
grid on;
```




```
disp(' ');
```

```
disp('7. Implement Inverse Filter');
```

```
7. Implement Inverse Filter
```

```
% 7. The inverse filter is implemented by the difference equation:
% x(n) + alpha*x(n-N) = y(n)
% Using filter(b, a, y) where a is the LHS (output x) and b is the RHS (input y)
% a coefficients (LHS): 1*x(n) + alpha*x(n-N) -> [1, 0, ..., 0, alpha]
a = zeros(1, N+1);
a(1) = 1;          % Coefficient for x(n)
a(N+1) = alpha;    % Coefficient for x(n-N)
% b coefficients (RHS): 1*y(n) -> [1]
b = 1;

disp('Inverse Filter Denominator a (for x(n) terms):');
```

```
Inverse Filter Denominator a (for x(n) terms):
```

```
disp(['a(1) = ', num2str(a(1)), ', a(', num2str(N+1), ') = ', num2str(a(N+1))]);
```

```
a(1) = 1, a(743) = 0.9
```

```
disp('Inverse Filter Numerator b (for y(n) terms): b = [1]');
```

```
Inverse Filter Numerator b (for y(n) terms): b = [1]
```

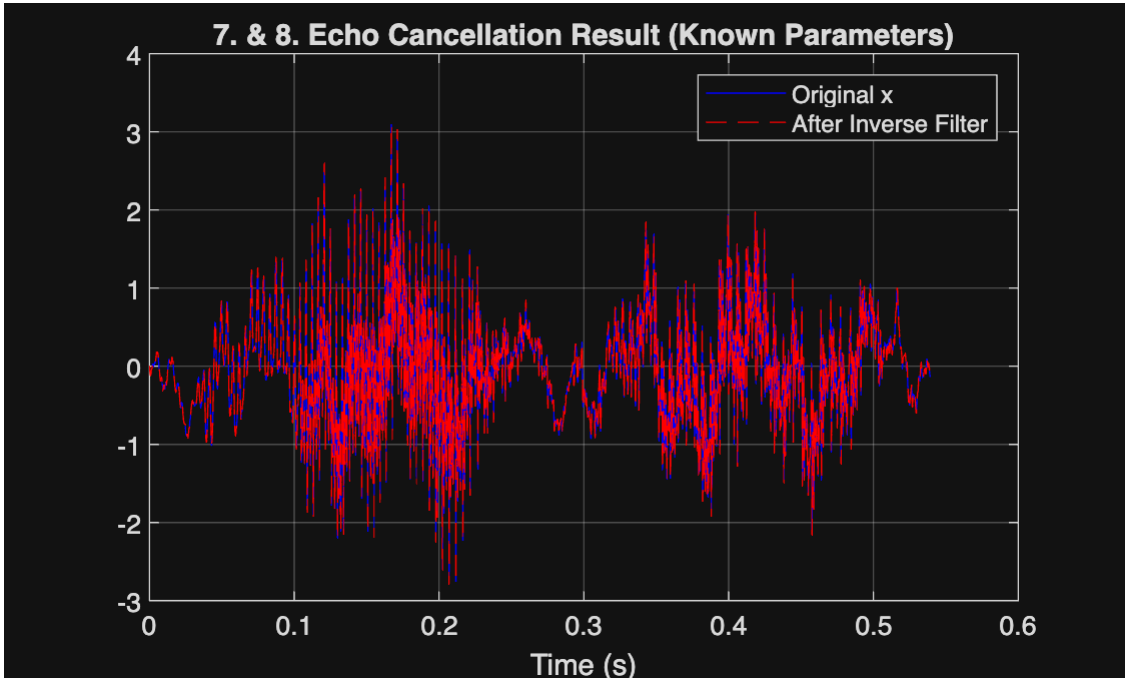
```
% Apply inverse filter to echoed signal y
filtered_x = filter(b, a, y);

% Trim the filtered signal back to the original length for comparison
filtered_x_trimmed = filtered_x(1:Nsamples);

% 8. Listen to the result
soundsc(filtered_x_trimmed, Fs);
disp('8. The echo-cancelled signal has been created and is plotted below.');
```

```
8. The echo-cancelled signal has been created and is plotted below.
```

```
% Exact Duration of the signal
duration_seconds = g_len/Fs;
disp(['Sampled Signal Duration (length(x)/Fs): ', num2str(duration_seconds), ' seconds']);
```

```
Sampled Signal Duration (length(x)/Fs): 1.1004 seconds
```

```
% Plot comparison (overlay)
figure;
plot(t_x, x, 'b');
hold on;
plot(t_x, filtered_x_trimmed, 'r--');
xlabel('Time (s)');
legend('Original x','After Inverse Filter');
title('7. & 8. Echo Cancellation Result (Known Parameters)');
grid on;
```




```
disp(' ');
```

```
disp('9.  Derivation of Autocorrelation ryy(k)');
```

```
9.  Derivation of Autocorrelation ryy(k)
```

```
% 9. Symbolic derivation: ryy(k) in terms of rxx(k)
% The Z-transform is Ryy(z) = Rxx(z) * H(z) * H(z^-1)
% Ryy(z) = Rxx(z) * (1 + alpha*z^(-N)) * (1 + alpha*z^N)
% Ryy(z) = Rxx(z) * (1 + alpha^2 + alpha*z^N + alpha*z^-N)

syms k N alpha % k = lag, N = delay, alpha = echo amplitude
% Define symbolic autocorrelation of x[n]
syms rxx(k)
% Echoed signal: y[n] = x[n] + alpha*x[n-N]
% Then the autocorrelation of y[n] is:
r_yy(k) = (1 + alpha^2)*rxx(k) + alpha*( rxx(k-N) + rxx(k+N) );
```

```
disp('Symbolic autocorrelation relation, ryy = ');
```

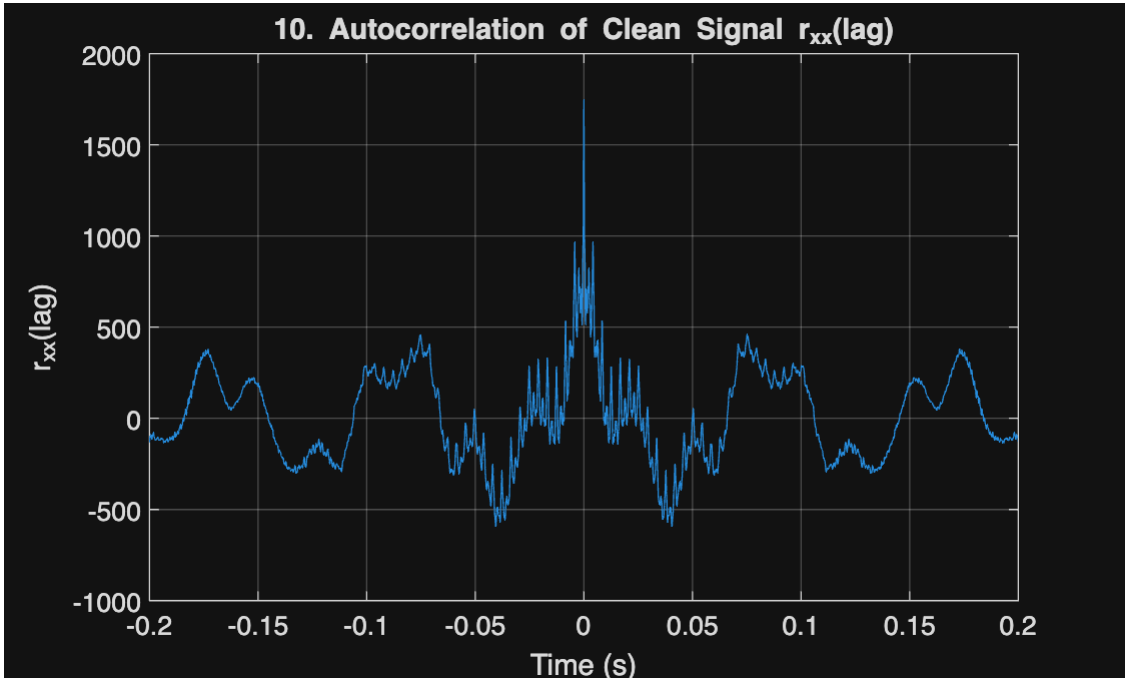
```
Symbolic autocorrelation relation, ryy =
```

```
pretty(r_yy)
```

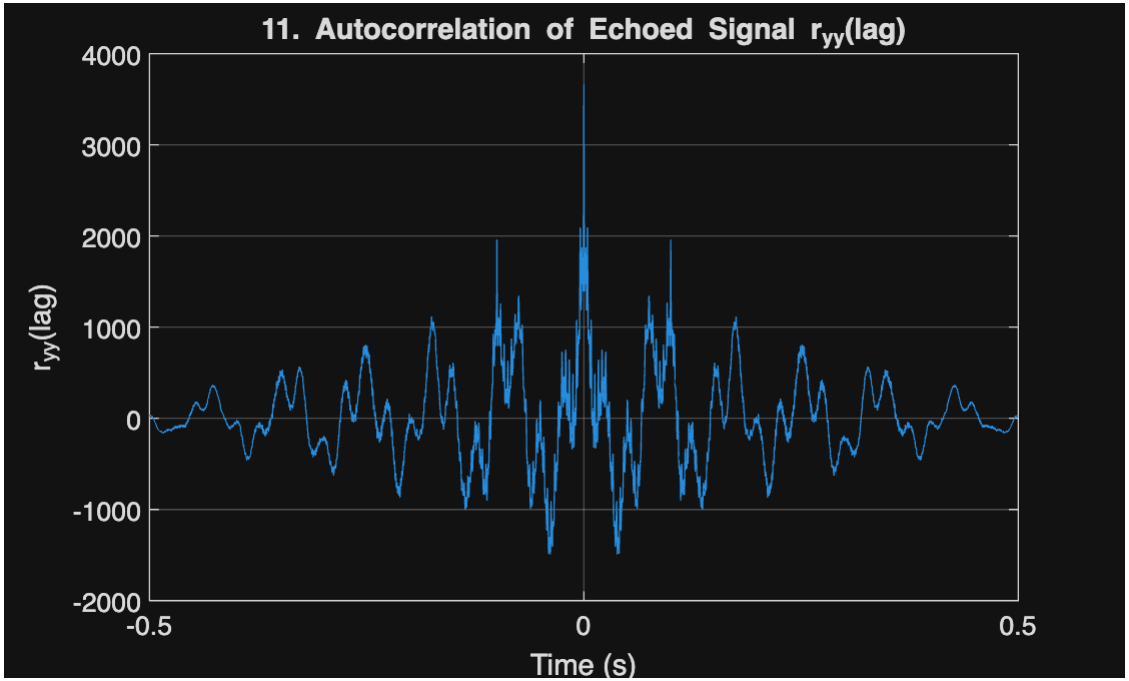
```
                2
rxx(k) (α  + 1) + α (rxx(N + k) + rxx(k - N))
```

```
disp(' ');
```

```
% 10. Compute and Plot rxx(n)
% Autocorrelation of x (echo-free signal)
rxx = conv(x, x(end:-1:1));
lags = -(Nsamples-1):(Nsamples-1);
time_lags = lags / Fs;

figure;
plot(time_lags, rxx);
xlabel('Time (s)');
ylabel('r_{xx}(lag)');
title('10. Autocorrelation of Clean Signal r_{xx}(lag)');
xlim([-0.2 0.2]); % Zoom in around center
grid on;
```




```
% 11. Plot ryy(n) and Determine N (Known Echo)
% Autocorrelation of y (echoed signal)
ryy = conv(y, y(end:-1:1));
lags = -(Nsamples-1):(Nsamples-1);
time_lags = lags / Fs;

ly = length(y);
lags_y = -(ly-1):(ly-1);
time_lags_y = lags_y / Fs;

figure;
plot(time_lags_y, ryy);
xlabel('Time (s)');
ylabel('r_{yy}(lag)');
title('11. Autocorrelation of Echoed Signal r_{yy}(lag)');
xlim([-0.5 0.5]);
grid on;
```




```
% 2. and 3. estN is the lag value (in samples) at the peak index based on
% graph above
est_t0 = 0.1000027;
estN = round(est_t0 * Fs);
disp(['11. From plot of ryy(n), the lag of the first side peak is approximately ', num2str(estN), '
samples (or ', num2str(estN/Fs), ' s)']);
```

```
11. From plot of ryy(n), the lag of the first side peak is approximately 742 samples (or 0.10003 s)
```

```
disp('This agrees with the known value N = 742 samples.');
```

```
This agrees with the known value N = 742 samples.
```

```
disp(' ');
```

PART 3: Estimating Unknown Echo Parameters

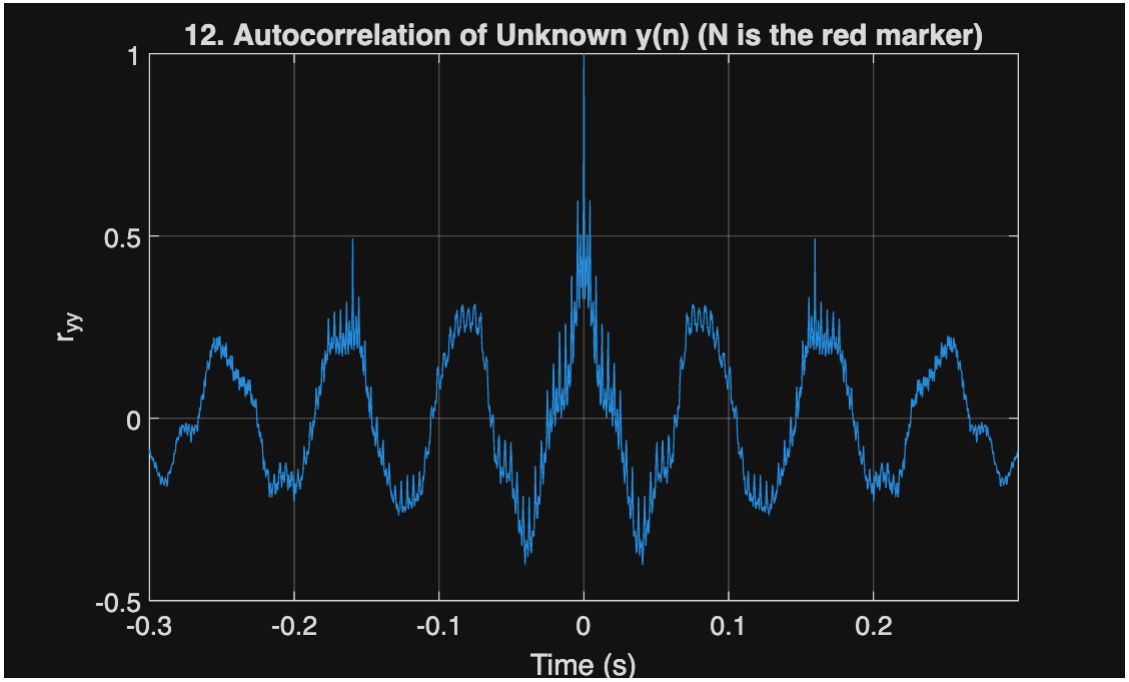
```
% 12. Estimating Unknown N from new data
load echo_data.mat; % Loads 'y' (new echoed signal) and 'Fs'

%Play sound data
soundsc(y, Fs);
Nsamples_unknown = length(y);
disp(' 12. Estimating N for Unknown Data ');
```

```
 12. Estimating N for Unknown Data
```

```
disp(['Loaded unknown echoed signal y with length: ', num2str(Nsamples_unknown), ' samples']);
```

```
Loaded unknown echoed signal y with length: 5185 samples
```

```

% Compute autocorrelation of the unknown signal
[ryy_unk, lags_unk] = xcorr(y, 'coeff'); % 'coeff' normalizes the autocorrelation
time_lags_unk = lags_unk / Fs;

% Plot autocorrelation and detected peak
figure;
plot(time_lags_unk, ryy_unk); hold on;
xlabel('Time (s)'); ylabel('r_{yy}');
title('12. Autocorrelation of Unknown y(n) (N is the red marker)');
xlim([-0.3, 0.3]);
grid on;
```




```
est_t0 = 0.159692;
estN = round(est_t0 * Fs);
fprintf('12. Estimated echo delay N ≈ %d samples (%.4f sec)\n', estN, est_t0);
```

```
12. Estimated echo delay N ≈ 1185 samples (0.1597 sec)
```

```
disp(' ');
```

```
disp('13. Estimating Unknown alpha and Echo Cancellation ');
```

```
13. Estimating Unknown alpha and Echo Cancellation
```

```
% 13. Estimating alpha via search (Optimization)
% Use the estimated N (estN) and search for the best alpha.
% The metric should minimize residual echo and noise.
alpha_vals = 0.1 : 0.01 : 2; % Search range for alpha
best_metric = inf;
best_alpha = 0;
```

```
for ai = alpha_vals

    % Construct the inverse filter denominator 'a'
    a_test = zeros(1, estN+1);
    a_test(1) = 1;
    a_test(end) = ai; % Use current alpha guess
    b_test = 1;

    % Apply inverse filter
    x_hat = filter(b_test, a_test, y);
    x_hat = x_hat(1:Nsamples_unknown); % Trim to original length

    % Optimization Metric: Minimize residual energy and high-frequency noise
    % High RMS often means speech is clear; high diff RMS often means noise/echo is present.
    metric = rms(x_hat) + 0.1*rms(diff(x_hat));

    if metric < best_metric
        best_metric = metric;
        best_alpha = ai;
        best_x = x_hat; % Store best reconstruction
    end
end

fprintf('13. Best Estimated Parameters: N = %d samples and α ≈ %.3f\n', estN, best_alpha);
```

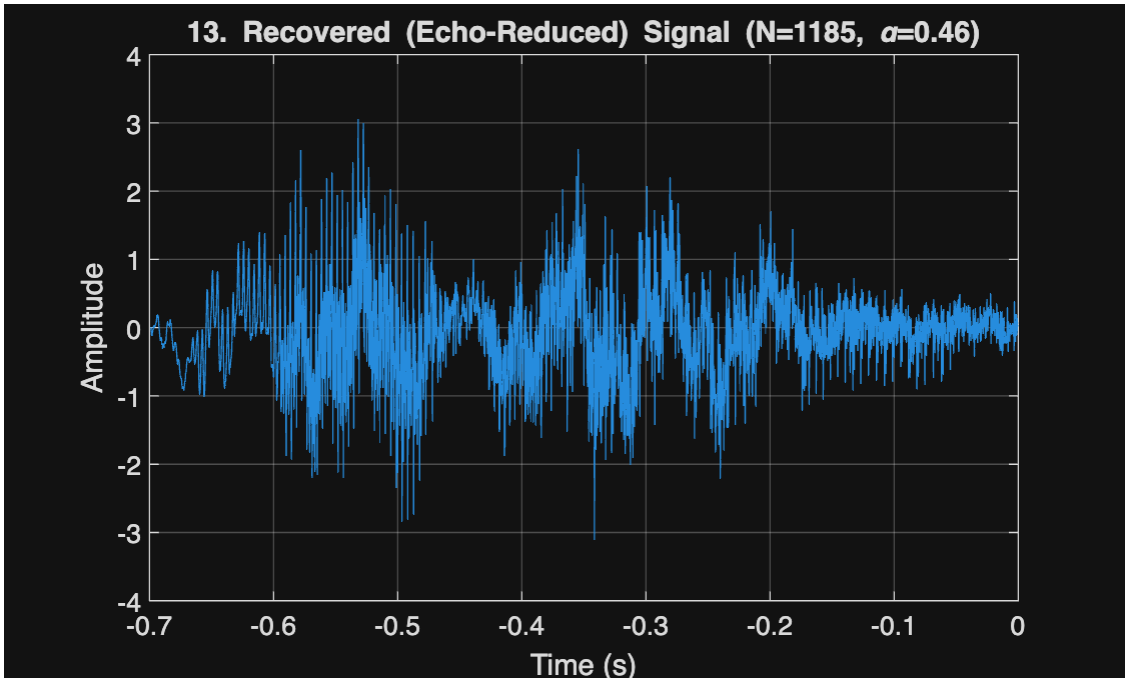
```
13. Best Estimated Parameters: N = 1185 samples and α ≈ 0.460
```

```
% Final Echo Cancellation using best N and alpha
a_final = zeros(1, estN+1);
a_final(1) = 1;
a_final(end) = best_alpha;
b_final = 1;

x_rec = filter(b_final, a_final, y);
x_rec = x_rec(1:Nsamples_unknown);

% Listen to recovered sound
soundsc(x_rec, Fs);
```

```
disp('The signal has been recovered using estimated parameters. ');
```

```
The signal has been recovered using estimated parameters.
```

```
% Plot recovered signal
figure;
plot(time_lags_unk(1:Nsamples_unknown), x_rec);
title(['13. Recovered (Echo-Reduced) Signal (N=', num2str(estN), ', \alpha=', num2str(best_alpha),
')']);
xlabel('Time (s)'); ylabel('Amplitude');
grid on;
```




```
disp(' ');
```

```
disp('14. Two-Component Echo (Discussion)');
```

14. Two-Component Echo (Discussion)

```
% 14. What if an echo has two components?
disp('y(n) = x(n) + α1*x(n-N1) + α2*x(n-N2)');
```

```
y(n) = x(n) + α1*x(n-N1) + α2*x(n-N2)
```

```
disp('System required for cancellation: **IIR Filter** with Transfer Function:');
```

```
System required for cancellation: **IIR Filter** with Transfer Function:
```

```
disp('G(z) = 1 / (1 + α1*z^(-N1) + α2*z^(-N2))');
```

```
G(z) = 1 / (1 + α1*z^(-N1) + α2*z^(-N2))
```

```
disp('Difference Equation: x(n) + α1*x(n-N1) + α2*x(n-N2) = y(n)');
```

```
Difference Equation: x(n) + α1*x(n-N1) + α2*x(n-N2) = y(n)
```

```
disp(' ');
```

```
disp('How to find echo parameters:');
```

```
How to find echo parameters:
```

```
disp('1. Estimate N1 and N2 (Delays): Compute the autocorrelation ryy(k). ryy(k) will exhibit four side
peaks (two symmetric pairs) corresponding to the lags ±N1 and ±N2. Identify the two largest positive
lags to find N1 and N2.');
```

```
1. Estimate N1 and N2 (Delays): Compute the autocorrelation ryy(k). ryy(k) will exhibit four side peaks (two symmetric pairs) corresponding
```

```
disp('2. Estimate α1 and α2 (Amplitudes): Perform a 2D search/optimization over a range of α1 and
α2 values. For each pair, construct the inverse filter and apply it to y(n). The pair (α1, α2) that
minimizes a suitable "cleanness" metric (e.g., residual energy/noise) in the recovered signal x_rec is
the best estimate.');
```

```
2. Estimate α1 and α2 (Amplitudes): Perform a 2D search/optimization over a range of α1 and α2 values. For each pair, construct the inverse
```